

¿Sustituirá la programación con IA a todos los programadores? Veamos la situación real del sector

Zhihu · 2026-09-19

Desde que a finales de 2025 y comienzos de 2026 la eficacia de la programación con IA mejoró notablemente, buena parte del trabajo de codificación de los programadores —casi todo, en algunos casos— ha sido sustituida por la IA. El fenómeno ha despertado pánico y desesperación entre los programadores. Al mismo tiempo, las empresas de IA, valiéndose de los avances en este campo, exageran sin límite las capacidades de la IA y proclaman que en uno o dos años todos los cuellos blancos irán a la calle. Los medios autogestionados se han sumado en tropel, difundiendo el pánico y fabricando ansiedad.

Sobre esta cuestión conviene examinar las cosas con seriedad. ¿Puede la programación con IA sustituir a todos los programadores? Y si no puede, ¿cuál es la tasa real de sustitución?

De hecho, en una respuesta que escribí hace unos meses («El primer tajo de la IA: degollar programadores —¿van a perder el empleo masivamente los programadores?») ya di mi diagnóstico: la IA no puede sustituir toda la mano de obra, y su tasa de sustitución de extremo a extremo no supera el 30%.

Cuando los sistemas de máquinas inteligentes centrados en la IA entren a gran escala en la producción, el trabajo de codificación manual desaparecerá en lo esencial. Claro que el desarrollo del código central, de arquitectura compleja o con requisitos de rendimiento muy altos, quizá siga necesitando una intervención humana intensa; pero el grueso del desarrollo de código ordinario será sustituido por completo por la IA. ¿A cuánta gente sustituirá en concreto? Según mis propios resultados de uso y los comentarios de otros en Zhihu, la mejora de eficiencia va del 20% al 50%; si la programación con IA se implanta de forma normal (hoy seguimos en la fase de ensayo masivo del sector) y se calcula en un 60% el peso de los desarrolladores puros, se recortaría en torno al 12%-30% de los puestos de programador. La implantación masiva de la programación con IA lleva ya al menos medio año y, sobre todo en China y Estados Unidos, casi todas las grandes tecnológicas exigen a sus empleados desarrollar con agentes de IA. Entonces, ¿qué resultados ha dado?

Según el informe de McKinsey^[1], para los programadores «que saben usar la IA», la productividad puede aumentar alrededor del 16%-30%, en líneas generales coincidente con mi diagnóstico anterior.

 Figure 1

Un estudio reciente^[2] también muestra que, aunque la programación con IA mejora enormemente la velocidad de generación de código, a la hora de reflejarse en la publicación final del producto, la eficiencia solo mejora alrededor de un 30%. Esto coincide de nuevo con el estudio de McKinsey citado arriba y con mi conjetura anterior.

Las herramientas de IA generativa para el desarrollo de software avanzan a velocidad asombrosa: en apenas unos años han pasado de la simple autocompletación de código a agentes de programación plenamente autónomos (coding agents). ¿Se traduce ese aumento de capacidades en un crecimiento real de la productividad a nivel de tarea? Y aunque se traduzca, ¿implica la mejora de la eficiencia por tarea un aumento del producto final? Estas preguntas son cruciales para entender el impacto económico de la IA. Mediante un estudio de eventos emparejados con desarrolladores, hallamos que, a medida que las herramientas de programación con IA se actualizan, su efecto de mejora de la productividad a nivel de tarea se hace cada vez más notable: la función de autocompletado elevó en torno a un 30% el número de envíos de código semanales; los agentes sincrónicos (sync agents) llevaron ese efecto acumulado a cerca del 180%, y los agentes asíncronos (async agents), a alrededor del 240%. Sin embargo, conforme se asciende a la fase de producción, esas ganancias se debilitan bruscamente: aunque los agentes sincrónicos aumentaron un 958% las líneas de código y un 86% las solicitudes de extracción (pull requests), el número de versiones finalmente publicadas solo creció un 20%. La actividad de programación en conjunto en GitHub muestra la misma pauta.

 Figure 2

No solo los estudios van confirmando los límites de la programación con IA: quienes de verdad usan agentes de IA, desarrolladores y directivos, lo sienten además de forma directa. La actitud de los programadores hacia la programación con IA ha salido del asombro inicial y, a través de la aplicación real en producción, va reconociendo poco a poco sus límites. En una entrada de blog^[3] de Cal Newport (profesor de informática en la Universidad de Georgetown) se relata, por ejemplo, el caso de un desarrollador veterano de Silicon Valley que, a comienzos de año, tras programar con Claude Code, multiplicó varias veces su productividad y exclamó «really, really good»; después de sufrir dos incidentes en producción causados por código de IA y de estar a punto de perder el empleo, quedó desencantado de la programación con IA, volvió a escribir a mano el código central y la reserva solo para pruebas, scripts y cosas por el estilo. Este desarrollador sostiene que la programación a la antigua es más lenta, pero fiable y con la calidad garantizada. El autor señala que no es un caso aislado: bastantes ingenieros del sector expresan opiniones parecidas.

¿Dónde está el problema de la programación con IA? Un programador lo aborda en su blog^[4], donde cuenta sus impresiones tras seis meses examinando código de IA. Sostiene que un agente de IA se parece a un empleado nuevo muy competente: escribe código que parece excelente, pero, por carecer de conocimiento del contexto del dominio, nunca tiene suficientemente en cuenta las diversas restricciones, los casos límite y las situaciones excepcionales, y además tiende a introducir problemas de seguridad. De hecho, un artículo reciente, «Constraint Decay: The Fragility of LLM Agents in Backend Code Generation»^[5], también aborda esta cuestión:

Nuestra investigación revela un fenómeno de «decaimiento de restricciones»: a medida que aumentan los requisitos estructurales, el rendimiento de los agentes cae de forma notable. De las tareas de referencia a las tareas con requisitos plenamente especificados, las configuraciones de mejor rendimiento bajaron en promedio 30 puntos porcentuales en la tasa de aprobación de aserciones, y algunas configuraciones más débiles cayeron incluso a niveles cercanos a cero. El análisis de sensibilidad por framework revela diferencias de rendimiento notables: los agentes funcionan bien en frameworks mínimos y explícitos (como Flask), pero en entornos muy dependientes de convenciones (como FastAPI o Django) su rendimiento medio se desploma. Por último, el análisis de errores muestra que los defectos de la capa de datos (por ejemplo, construcción errónea de consultas y excepciones de ejecución del ORM) son la raíz principal de estos problemas. El estudio subraya que satisfacer a la vez los requisitos funcionales y los estructurales sigue siendo un desafío clave y no resuelto para los agentes de programación.

El CTO de una empresa india también lo señalaba en un artículo reciente^[6], donde afirma que el código escrito por la IA tiende a dar problemas en la operación real de producción porque «en los sistemas grandes, el código debe ajustarse a los patrones de arquitectura, las cadenas de dependencias, las reglas de negocio, los requisitos de cumplimiento, los controles de seguridad y los procesos de publicación. Si el código generado por la IA ignora estas restricciones, el centro de gravedad del trabajo se desplaza del desarrollo a la posterior reparación y rectificación, lo que a su vez prolonga los ciclos de revisión de código, deja una cobertura de pruebas insuficiente y provoca riesgos de seguridad no detectados, fallos de integración y defectos en el entorno de producción». Quienes tienen experiencia real programando con IA comprenden que estos problemas son, en efecto, generalizados.

La IA escribe código deprisa, pero, por los problemas arriba indicados, los desarrolladores tienen que examinar con lupa el código generado por IA antes de integrarlo en el repositorio. De ahí se derivan dos problemas. Primero, la velocidad de revisión del código es muy inferior a la de su generación, lo que reduce mucho la mejora de eficiencia que aporta la IA. Segundo, los programadores nuevos carecen de ocasiones de escribir código con sus propias manos y no pueden mejorar su capacidad; cuando la vieja generación de programadores sea despedida o se jubile, la velocidad de revisión se resentirá aún más y se llegará incluso a no detectar los problemas.

Más aún: algunas empresas depositan grandes esperanzas en la programación con IA y comprimen mucho los ciclos de desarrollo, de modo que los programadores no tienen tiempo material para revisar el código con cuidado. Para el desarrollador, la empresa tampoco es suya: si el jefe exige rapidez, uno revisa por encima o incluso integra sin revisar. Si luego falla en producción, ya se buscará otro sitio. Sea por fatiga de revisión o por revisión insuficiente, el resultado final es que el código que llega al entorno de producción esconde un montón de BUG y de «malos olores» de código, y quienes luego mantienen ese software no conocen el código en realidad. Toda esa deuda técnica da lugar a la llamada «brecha de mantenibilidad» (The Maintainability Gap)^[7].

La programación asistida por IA no deja de crecer con ímpetu, y los desarrolladores más entusiastas producen volúmenes de código sin precedentes. Sin embargo, un análisis de cuatro años de datos de cambios en el código muestra que los indicadores de mantenibilidad van a la baja: las llamadas a funciones entre archivos (que reflejan la reutilización de código) disminuyeron un 35%; en comparación con los niveles de 2022, las migraciones de líneas de código en refactorizaciones cayeron un 70%, y el trabajo de mantenimiento del código heredado de largo plazo, un 74%. Al mismo tiempo, observamos tendencias al alza preocupantes: el comportamiento de «copiar y pegar» dentro de una misma confirmación aumentó un 41%, la duplicación de bloques de código, un 81%, las estructuras de código que enmascaran errores, un 47%, y la tasa de pérdida de código (churn) en dos semanas, un 15%. Aunque la producción de código crece en efecto mucho, la deuda técnica que se acumula con ella no es en absoluto desdeñable; y esa deuda recae sobre todo en los desarrolladores que aún no se han dado cuenta de que ciertas pautas de codificación ponen en peligro la mantenibilidad del repositorio a largo plazo.

Sobre la deuda técnica, el artículo «Debt Behind the AI Boom»^[8] señala, mediante un estudio a gran escala, que «los malos olores de código son el tipo de problema más frecuente, con hasta un 89,3%»; «en las confirmaciones generadas por los distintos asistentes de programación con IA, más del 15% introdujo al menos un problema. Y lo que es más importante: de los problemas introducidos por la IA, el 22,7% sigue presente en la última versión del repositorio». «Estos hallazgos indican que el código generado por la IA puede acarrear costes de mantenimiento de largo plazo a los proyectos de software reales, y ponen de relieve la necesidad de reforzar el aseguramiento de la calidad en el desarrollo asistido por IA».

 Figure 3

Procedente de: «Debt Behind the AI Boom»

Y el artículo «Faster Code, Deeper Debt?»^[9] indica que «los LLM tienden a agravar las formas tradicionales de deuda técnica (en particular la deuda de código, de diseño y de documentación) y, a la vez, introducen nuevas formas de deuda propias de los LLM. Es de notar que hemos identificado una «deuda de integración rápida»: sacrificar la calidad en aras de la velocidad hace que el código generado a toda prisa desencadene reacciones en cadena, generando deuda de gobernanza y mayores costes de mantenimiento a largo plazo. Otras categorías emergentes de deuda son la deuda de prompts, la deuda ética, la deuda de datos y la deuda de trazabilidad, que reflejan los desafíos específicos que trae consigo la aplicación de los LLM».

La programación con IA no solo acumula deuda técnica: introduce además otras deudas. En su artículo «From Technical Debt to Cognitive and Intent Debt»^[10], la informática Margaret-Anne Storey plantea que, más allá de la deuda técnica, «a medida que la velocidad con que la inteligencia artificial genera código supera la velocidad con que los equipos lo comprenden, se están acumulando dos formas de "deuda" a menudo subestimadas»: distingue, primero, la «deuda cognitiva» (cognitive debt), esto es, la pérdida del consenso dentro del equipo; y segundo, la «deuda de intención» (intent debt), es decir, la falta de la justificación lógica externalizada que desarrolladores y agentes de inteligencia artificial necesitan para manejar el código con seguridad. Dicho en lenguaje llano: el uso generalizado de la programación con IA puede acabar con que nadie en todo el equipo tenga claro qué está haciendo. Hoy la atención del sector sigue centrada en la deuda técnica; estos problemas más profundos todavía no se han manifestado del todo.

Llegados a este punto, creo que el lector ya tiene su respuesta. En muchas respuestas anteriores (por ejemplo, «¿Crees que la programación con IA sustituirá de verdad a los programadores en el futuro?» y «¿Qué pensar de la predicción del responsable de IA de Microsoft: en los próximos 18 meses la IA sustituirá por completo a la mayoría de los empleos de cuello blanco?») he repetido una y otra vez una idea: la IA transformará profundamente la forma de trabajar de los cuellos blancos, pero no puede eliminar a todos los programadores, y mucho menos dejar en la calle a todos los cuellos blancos. La realidad ha demostrado que mi idea es correcta.

A medida que se desvanece el mito de que la IA sustituirá a todos los cuellos blancos, el mercado de capitales empieza a mostrar signos de reventón de la burbuja. Partiendo del supuesto de una mejora de eficiencia del 30% en el desarrollo de software, ya deduje antes que la burbuja de la IA existe realmente y es mayor que la burbuja de internet (véase «¿Qué tan grande es la burbuja de la IA? ¿Superará con creces una crisis de la burbuja de la IA a la crisis financiera de 2008?»). Últimamente, la tesis del exterminio de la humanidad por la IA que sueltan los consejeros delegados de las grandes empresas de modelos de IA no es más que la locura final antes de su perdición.

Referencias

- [illegible]

Notas

[1] The AI revolution in software development

中文原文: The AI revolution in software development

<https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-ai-revolution-in-software-development>

[2] Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools.

中文原文: Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools.

[3] On AI Coding and Its Discontents

中文原文： On AI Coding and Its Discontents

<https://calnewport.com/on-ai-coding-and-its-discontents/>

[4] I Spent 6 Months Reviewing AI Code. Here's the Ugly Truth

中文原文： I Spent 6 Months Reviewing AI Code. Here's the Ugly Truth

https://medium.com/@haseeb_sohail/i-spent-6-months-reviewing-ai-code-heres-the-ugly-truth-b81fe33e0b25

[5] Constraint Decay: The Fragility of LLM Agents in Backend Code Generation

中文原文： Constraint Decay: The Fragility of LLM Agents in Backend Code Generation

<https://arxiv.org/pdf/2605.06445>

[6] The Productivity Illusion at the Heart of Enterprise AI Coding

中文原文： The Productivity Illusion at the Heart of Enterprise AI Coding

<https://cacm.acm.org/blogcacm/the-productivity-illusion-at-the-heart-of-enterprise-ai-coding/>

[7] The Maintainability Gap: AI Code Quality in 2026

中文原文： The Maintainability Gap: AI Code Quality in 2026

https://www.gitclear.com/the_ai_code_quality_maintainability_gap

[8] Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild

中文原文： Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild

<https://arxiv.org/abs/2603.28592>

[9] Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development

中文原文： Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development

<https://arxiv.org/abs/2606.14796>

[10] From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI

中文原文： From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI

<https://arxiv.org/abs/2603.22106>

Este artículo fue publicado originalmente en Zhihu (en chino): <https://zhuanlan.zhihu.com/p/2084561490930045240> Autor: 最最遥远的路 · Perfil de Zhihu: <https://www.zhihu.com/people/zui-zui-yao-yuan-de-lu-32/posts>

本文原发表于知乎专栏/回答（中文）： <https://zhuanlan.zhihu.com/p/2084561490930045240> 作者： 最最遥远的路： <https://www.zhihu.com/people/zui-zui-yao-yuan-de-lu-32/posts>